

Efficient Visual Question Answering under a 3B Parameter Budget

Yookyung Youn | Korea University

Technical report | SCPC 2025 AI Challenge, 1st Place | Documentation edition: September 2026

Abstract

The SCPC 2025 AI Challenge required multiple-choice visual question answering with fewer than three billion parameters loaded for inference. I independently designed and implemented a solution by compressing InstructBLIP-Flan-T5-XL, stabilizing the compressed model through knowledge distillation, and adding custom LoRA adapters. This report develops the reasoning behind that solution: choosing an architecture whose components could be investigated separately, testing where compression was affordable, revising activation-based assumptions after non-monotonic masking results, and distinguishing recovery from further task adaptation. The final model contains 2,986,746,208 parameters, approximately 25.8% fewer than the original. Reported public weighted accuracy increased from 82.83% after pruning to 85.73% after distillation and 86.59% after LoRA. The solution received first place in the AI Challenge. Alongside these results, I discuss qualitative language-generation degradation and the limits of task accuracy as evidence of retained capability. This retrospective report distinguishes archived implementation details from recalled experimental observations and credits the existing methods on which the solution builds.

1. Setting, Ownership, and Evidence

1.1 A task and constraints, with an open solution strategy

Each example contains an image, a question, and candidate answers. The objective is to select an answer using visual information and relevant knowledge. I worked as an individual undergraduate participant, taking responsibility for model and dataset selection, compression design, training, evaluation, implementation, and final submission. The central question was which eligible starting point could become a competitive, compliant system within the resources I could actually use.

The rules required fewer than 3B total parameters across all models loaded during inference, including models loaded sequentially. Eligible pretrained weights had to have been publicly released by December 31, 2023; external data had to have been released by June 10, 2025, with licenses meeting the competition requirements. Remote model APIs were prohibited [1]. This was a parameter-count constraint: reduced numerical precision or a small number of activated parameters would not by itself establish compliance.

1.2 The effective development window

I recall concentrating implementation and experiments into approximately two to three weeks because the competition overlapped with end-of-semester commitments. This describes my effective development window, not the official duration of the event. GPU access was paid for personally through on-demand rentals. The cost of a failed run affected which hypotheses I could test and how much evidence I sought before committing to longer training.

These constraints encouraged inexpensive diagnostics, incremental changes, and reuse of pretrained capabilities. They also made stopping decisions consequential. A plausible alternative could still be a poor use of the remaining time if its alignment, training, or debugging costs were difficult to estimate. The choices below should be understood within that setting.

1.3 How this account was reconstructed

This report was reconstructed in September 2026 from archived competition materials, inspected notebooks, and my recollection. Exact implementation settings and tabulated scores are attributed to the archive. Explanations of why I tried an approach, and qualitative comparisons without retained measurements, are retrospective recollections. The presentation follows dependencies between decisions, rather than claiming a complete dated sequence of every run.

My contribution is the independent formulation, experimental development, structural implementation, and integration of this solution. I arrived at several design questions by reasoning about the task and architecture before looking for a matching method. In the FFN investigation, I subsequently found Wanda's activation-and-weight principle and adapted it. This account does not claim invention of pruning, distillation, LoRA, or a new general theory. Section 9 separates existing methods used in the system from literature consulted later to contextualize the experience.

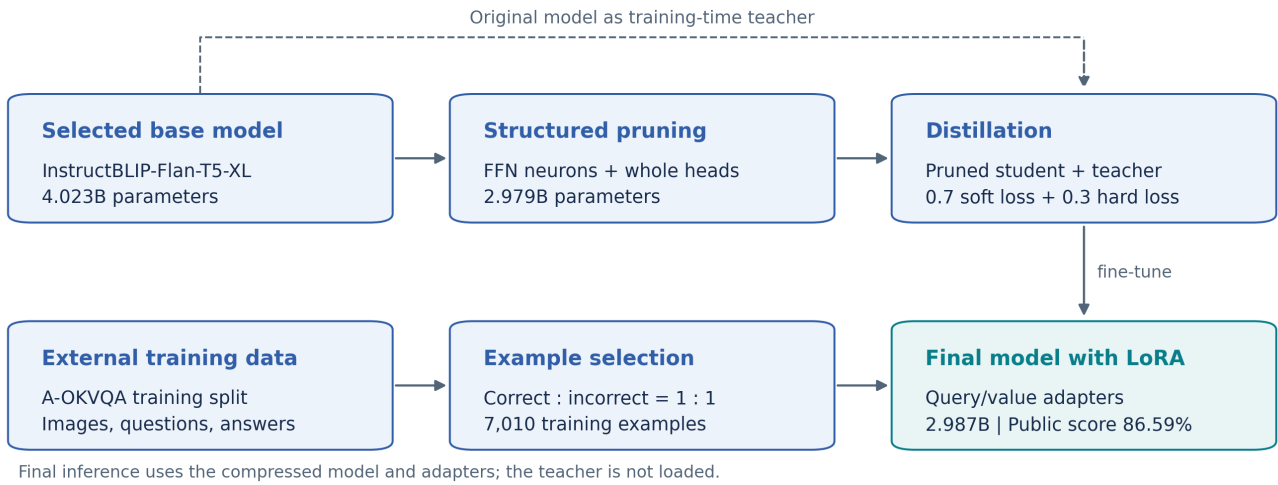


Figure 1. Final system. FFN neurons and whole attention heads are structurally removed. The original model supervises the student during training. Final inference uses the compressed student with adapters. This diagram shows the retained solution; the investigations leading to it are developed below.

2. Selecting a Model I Could Investigate

2.1 The eligible model pool did not provide an obvious answer

My initial intention was to start with a capable larger model and prune it. Among the eligible candidates I explored, I recall an awkward set of trade-offs: some already fit below 3B, some exceeded the limit without an obvious enough task advantage over smaller alternatives, and others would require much more aggressive compression. These were impressions from the candidates and settings I tried. The surviving record does not support an exhaustive model-ranking table or a statistically significant comparison of architecture families.

I also considered assembling separately pretrained components and adapting a more unified architecture. The problem was to find both useful starting capability and an experimental path whose failures I could diagnose. A high initial score alone did not tell me how manageable the compression problem would be.

2.2 Why I deferred assembling separate components

Combining a strong encoder with a separately pretrained language component appeared attractive because it would let me choose the size of each part. My concern was that separately learned representations would not automatically be compatible. A bridge could reconcile tensor dimensions without necessarily preserving the meaning or behavior expected by the receiving component.

I anticipated two costs. First, learning a useful bridge might require more data and GPU time than I could afford. Second, unfreezing the components to force alignment could disturb their useful pretrained behavior and create another recovery problem. At the time I worried that alignment might approach pretraining-scale effort. That was a conservative risk estimate, not a demonstrated lower bound on the required data.

I therefore deferred this route within the competition window. The decision was that uncertain alignment costs made it less predictable than modifying a model whose components already worked together. It was not a finding that component assembly is generally ineffective. Section 9 revisits the data-cost assumption using relevant research.

2.3 Why InstructBLIP-Flan-T5-XL made component-level experiments practical

For some more unified architectures I explored, pruning outcomes felt difficult to localize. A change could affect tightly coupled functions, and the resulting losses were irregular. I wanted to ask more focused questions about where information was transformed and which component had changed.

InstructBLIP-Flan-T5-XL offered an image encoder, a Q-Former, and a T5 language model with separate encoder and decoder components [3]. That decomposition created more opportunities to intervene in one part and observe the consequence. The Q-Former connects visual features to the language model; it is distinct from the connection between the T5 encoder and decoder.

The choice combined existing task capability with experimental tractability. I accepted a substantial compression requirement because the architecture gave me a clearer way to investigate it. The archived baseline contains 4,022,969,088 parameters and achieves 79.00% accuracy on a 1,000-example A-OKVQA validation subset. That is a local baseline, not a competition public score.

3. From Architectural Intuition to Pruning Experiments

3.1 Why I expected some interventions to be more costly

Although the final system prunes decoder FFNs and attention heads, the initial candidate set included the image encoder, language encoder, Q-Former, decoder representations, FFNs, and attention heads.

My first hypothesis concerned information flow. If an early transformation corrupts a representation used by many subsequent computations, later components must operate on that altered input. I expected more opportunities for error propagation than from a localized change near the output. This was the intuition behind my informal description of a snowball effect. It suggested where to investigate sensitivity; it did not establish a monotonic relationship between depth and pruning tolerance.

My second concern was the interface between modalities. The Q-Former appeared especially consequential because it mediates visual information entering the language model. Disrupting that interface could make otherwise capable components less useful together. I therefore wanted evidence of sufficient savings and tolerance before altering it.

I recall testing both the image encoder and language encoder and seeing severe, irregular performance losses even after changes I considered small. That supported redirecting limited experiments toward the decoder. Full numerical records for those trials have not been recovered. The Q-Former concern remains an architectural hypothesis here; I do not assign it a controlled pruning result. These observations explain the project-specific selection without implying that encoders or bridges should never be pruned.

3.2 Choosing the data before choosing the score

I wanted to retain computation useful for the competition task. That made calibration data part of the pruning design. I selected A-OKVQA for its combination of visual questions, world knowledge, and a multiple-choice formulation [4]. I recall considering OK-VQA too, but the answer-selection format was an additional reason to prioritize A-OKVQA.

The question was: when the model performs a similar task, which parts of its computation participate, and what happens when they are suppressed? Activation was attractive because it was observable with modest additional computation. It supplied an initial signal, rather than a direct measurement of semantic importance.

3.3 Why activation clustering changed my assumptions

I investigated hidden-state masking using activation statistics and clustering. I recall beginning with relatively low-activation groups, expecting smaller performance losses. Many values were similarly small, however, so a coarse grouping could combine dimensions with very different effects. I subdivided groups to obtain finer candidates within the compute budget.

Masking did not produce a clean, monotonic relationship between activation magnitude and damage. Some low-activation groups caused substantial losses; other combinations changed little, and some improved performance. Low activation on a limited sample could not establish that a dimension was dispensable.

I considered whether some dimensions supported useful background information without becoming strongly active in those examples. I also considered interactions: removing one dimension could have a different effect depending on which others remained. I imagined complementary or mutually constraining groups. Those were hypotheses motivating further tests, not evidence identifying specific neurons as storing background knowledge or demonstrating a particular mechanism.

I then used a greedy process, evaluating candidate masking choices in the context of choices already made. Activation continued to propose candidates, and I tried additional combinations to avoid relying entirely on an initial ranking. The change in reasoning was to treat usefulness as conditional on the retained model. The search was neither exhaustive nor guaranteed to find a global optimum, but it directly tested the assumption that a static ranking had been missing.

3.4 Why I removed hidden-state masking from the final design

I recall a region in which additional masking changed performance relatively little, followed by sharp losses beyond some extent. The exact threshold is not recoverable from memory. The practical issue was that further intervention yielded too little additional parameter benefit for the associated task-performance risk.

Initially I investigated FFNs with the hidden-state intervention already present because I expected good removal combinations to depend on the current architecture. Once FFN compression supplied most of the useful savings, I reconsidered whether hidden-state removal still earned its cost. I chose the simpler final path based on FFN neurons and attention heads.

This was a substantive design revision. An idea could be worth investigating without belonging in the submission: these experiments exposed non-monotonic sensitivity and the importance of combinations. In the final architecture, the language model's input/output hidden width remains 2,048. Earlier hidden-state masking is an exploratory branch, not an additional final pruning stage.

4. Allocating the Parameter Savings

4.1 Why FFNs came before attention heads

I compared removal units by both their parameter savings and the behavior they might disturb. I thought of attention heads as potentially different ways of relating parts of the input. This was a heuristic interpretation, not a literal one-head-one-function claim, but it made me reluctant to obtain the entire reduction by removing many heads at once.

Decoder FFN intermediate neurons offered finer candidates with substantial aggregate savings. My accounting suggested a staged allocation: obtain a large part of the reduction from FFNs, then use head removal to meet the remaining budget. This ordering emerged from parameter accounting and experiments; the final pair of components had not been the only candidates considered.

4.2 From an activation idea to a Wanda-inspired criterion

Activation alone had proved insufficient. I wanted to incorporate how strongly a neuron's activity could affect subsequent computation. The output projection provided an inexpensive additional observation. A neuron with both small observed activation and small outgoing influence seemed a more plausible candidate than one assessed through either quantity alone.

When looking for a formal criterion, I encountered Wanda's weight-and-activation principle [5]. It supported the direction I had been considering, and I adapted it to the neuron groups and measurements I could afford. This is where a specific literature method entered the FFN design. I do not claim an independently proven formula or an exact implementation of Wanda's original elementwise criterion.

For intermediate neuron j in decoder layer l , the archived score is:

$$\text{score}(l, j) = \text{mean}(\text{abs}(\text{gated_activation}(l, j))) * \text{L2_norm}(\text{wo}(l)[:, j])$$

The mean uses activations collected by the calibration routine during generation on the first 250 A-OKVQA training examples. It is a proxy for contribution on sampled inputs, not a bound on information loss. Its value is that it narrows an otherwise costly search while remaining cheap to compute.

4.3 What the final FFN operation actually removes

The decoder originally has 24 gated FFNs, each with intermediate width 5,120 and input/output width 2,048. I globally ranked intermediate neurons across decoder layers and selected the lowest-scoring 95% for removal. Candidate masking suppressed entire columns of the output projection. Final compaction removed the corresponding rows of both input projections, wi_0 and wi_1 , and columns of wo , rebuilding smaller dense layers.

This is neuron-level structured pruning. The FFN intermediate widths shrink while the model-facing hidden width stays fixed. The 95% rate applies across decoder FFNs in aggregate, not separately to every layer or to the whole model. The recorded FFN count changes from 754,974,720 to 37,748,736 parameters.

I recall inspecting layer-level effects and seeing non-monotonic accuracy changes as removal increased. The retained rate was a practical trade-off among tested settings and the budget, not a universal optimum. My aim was to limit damage to sensitive computation. The

implementation establishes a global score ranking with different retained widths per layer; it does not establish a separately calibrated equal-damage allocation across layers.

4.4 From a dependency hypothesis to head-similarity analysis

After substantial FFN removal, I expected some attention computations to become less useful or partly redundant. I considered whether a head might lose utility when associated FFN computation was removed. This was a reason to investigate dependencies, not an implemented mapping between specific heads and FFN neurons.

The actual signal was attention-map similarity. If two heads behaved similarly on inspected inputs, I hypothesized that removing one could preserve more useful behavior than removing heads with different patterns. Similarity narrowed the candidate set; task evaluation still had to decide. Similar maps do not imply interchangeable value projections or identical downstream contributions.

The inspected notebook computes cosine similarities on the original unpruned model, using 500 A-OKVQA training images and a generic image-description prompt. Greedy candidate selection and approximately 60 removal configurations were evaluated on a 100-example validation subset. The workflow includes head-only tests followed by evaluation combined with FFN pruning. The implementation therefore differs from my recalled possibility of scoring explicit head-FFN connections: it uses original-model attention similarity, not Wanda-based head scoring or a measured head-to-FFN association.

4.5 The retained structure

The zero-based removed indices are [1, 3, 4, 8, 10, 11, 12, 14, 17, 21, 24, 25, 26]. The same set is applied to decoder self-attention and cross-attention across 24 layers. Each module retains 19 of 32 heads.

Compaction removes the associated rows of query/key/value projections and columns of the output projection. At 64 dimensions per head, internal width changes from 2,048 to 1,216 while input/output hidden widths remain fixed. The recorded attention count changes from 805,306,368 to 478,150,656 parameters.

The resulting model has 2,978,586,976 parameters. Reported A-OKVQA validation accuracy falls from 79.00% to 76.60% on 1,000 examples, while separate public weighted accuracy is 82.83%. Meeting the size constraint left a concrete next problem: recover useful behavior in the compliant architecture.

5. Knowledge Distillation as Stabilization

5.1 Why the original model was a natural teacher

I approached the next stage as recovery. The student inherited the original weights but had lost computation through pruning. I wanted supervision that could restore behavior it had previously supported, without enlarging its architecture.

The original unpruned InstructBLIP-Flan-T5-XL was a natural teacher because its behavior was closely related to the student's starting point. A larger teacher might know more, but that did not guarantee its behavior would be easier for the compressed student to imitate. Under a limited GPU budget, useful recovery per run mattered more than teacher size alone.

I recall trying a larger InstructBLIP-Flan-T5-family teacher. It improved fit to A-OKVQA training data but transferred less well to A-OKVQA validation and the public evaluation than the original teacher. The complete run and exact larger checkpoint have not been recovered, so this remains a qualitative recollection rather than a numerical teacher-size ablation. It informed the selected teacher without establishing a general advantage for smaller teachers.

5.2 Why I preferred output supervision to richer targets

I considered matching hidden states to restore internal behavior. My concern was that the pruned student might not efficiently reproduce detailed representations from the unpruned teacher. Hidden-state transfer also required GPU memory. I recall slower or less useful recovery and memory limitations, but a complete matched comparison is unavailable. Capacity mismatch was a working explanation, not a cause isolated from memory, implementation, and optimization effects.

The final procedure supervises answer-token distributions and target answer text. This directly serves the stabilization objective without requiring the student to reconstruct every teacher representation. Output-level supervision was the practical choice I retained for the task and resources.

I also explored explanation-based training after observing poorer language generation. Learning explanations might recover behavior that answer-only supervision neglected. Archived experiments include teacher-generated rationales and recovery training; I recall language improvement but little additional answer-selection benefit relative to GPU cost. I omitted that branch from the final solution. Some auxiliary quality metrics in these prototypes were calculated outside the differentiable graph, so they should not be reported as additional gradient-based objectives successfully optimized by the student.

5.3 Verified stabilization settings

The final stabilization notebook mixes soft-target distillation with hard-label cross-entropy [6]:

$$L = 0.7 * L_KD + 0.3 * L_CE$$

$$L_KD = T^2 * KL(\text{softmax}(\text{teacher_logits} / T) || \text{softmax}(\text{student_logits} / T)); T = 3$$

The implementation uses kl_div with reduction='batchmean'. Teacher and student receive images, questions, and target answer sequences during training. Hard labels are answer text, not merely A/B/C/D. This is token-distribution matching under teacher forcing. The KL call operates on the logits tensor without a separate padding mask; it should not be described as a masked token-average objective.

Setting in the final stabilization notebook	Value
Teacher and student	Original InstructBLIP-Flan-T5-XL in evaluation mode; structurally pruned student
Data and internal split	A-OKVQA train, 17,056 examples; 90% training / 10% validation, seed 42
Temperature and mixture	T = 3; 0.7 KD + 0.3 cross-entropy
Epochs and learning rate	3; 5e-6
Batch size and accumulation	1; 4 steps
Schedule and clipping	Linear schedule with 10% warmup; gradient norm clipped at 1.0
Checkpoint selection	Lowest internal validation cross-entropy, evaluated every 1,000 optimizer steps

These settings come from the final notebook, whose execution outputs are not retained. A separate development notebook contains a 0.65/0.35 loss mixture and completed logs. Multiple KD settings are therefore supported, but a controlled comparison linking every configuration to public submissions is unavailable. The 85.73% score is the archived KD-stage result, not a new reproduction of that notebook.

Distillation keeps the pruned architecture and improves the reported public score by 2.90 percentage points. The teacher is used only during training. This fulfilled the stabilization role

I had assigned to the stage and allowed me to consider using the remaining parameter allowance for further adaptation.

6. LoRA as Controlled Task Adaptation

6.1 Why I added capacity while freezing the backbone

After stabilization, the model still had room below 3B. LoRA offered additional trainable capacity whose parameter count could be controlled through its rank and target projections [7]. I wanted to improve task performance while preserving the recovered backbone.

My concern with full-model fine-tuning was the mismatch between broad pretraining and the much smaller A-OKVQA training set. Updating all compressed weights could specialize the system too strongly and disturb behavior that stabilization had restored. I recall such degradation in full-update trials and interpreted it as forgetting or over-specialization. The retained evidence does not isolate catastrophic forgetting as the unique cause. Freezing the backbone was my practical response to that risk.

6.2 Making adapters work with the modified architecture

Pruning had changed attention projection dimensions. I recall errors in the library adapter route I initially tried. The archived implementation addresses this by wrapping actual compressed linear layers and deriving adapter dimensions from their `in_features` and `out_features`.

The wrapper adds two trainable low-rank matrices to the frozen original linear transformation, scaled by α/rank . The selected rank is 16, α is 32, and adapter dropout is 0.1. It targets query and value projections in T5 encoder self-attention and decoder self-attention/cross-attention, for 144 wrapped projections. Base parameters remain frozen while adapter parameters train. The contribution here is making an existing method compatible with the structurally altered model.

The adapters add 8,159,232 parameters, bringing the final count to 2,986,746,208. That leaves 13,253,792 below the 3B boundary. Accounting for this added capacity was necessary for inference compliance as well as training.

6.3 Correct and incorrect examples served different purposes

Task data were limited. Training only on already-correct examples seemed unlikely to address errors efficiently and could reinforce familiar behavior. Training only on mistakes posed a different risk: selecting examples by failure would overrepresent the particular difficulties of the current model.

I wanted the update to learn from errors while continuing to encounter behavior worth retaining. I separated A-OKVQA training examples using the preceding model's predictions and sampled equal numbers from the correct and incorrect groups. The inspected code reads saved incorrect-question identifiers, samples each group with seed 42, and shuffles the mixture. The recorded training set has 7,010 examples.

I recall comparing several ratios and finding 1:1 most useful in local A-OKVQA and public-score comparisons. The code verifies the final ratio, but the full sweep has not been recovered. Equal sampling is therefore a selected setting for this project, not a universal optimum. It deliberately changes the distribution to address preservation and correction; it does not make that distribution representative of every VQA setting.

6.4 When more training stopped transferring

Initially, I recall broadly consistent local and public-score trends. With longer training, A-OKVQA accuracy continued to improve while the public score declined. That made further improvement on the development domain insufficient justification for continuing training.

Training used external training examples, not the competition evaluation data. The divergence suggested increasing specialization to A-OKVQA, but it did not isolate the mechanism. The exact A-OKVQA split and matched checkpoint values for the extended-training comparison have not been recovered; this report does not relabel it as a verified official-validation learning curve.

The retained LoRA notebook reloads a step-1,000 checkpoint. That records the selected checkpoint, not a general early-stopping rule. Final public weighted accuracy is reported as 86.59%, a further gain of 0.86 percentage points. Because public feedback informed development choices, it remains development feedback rather than an untouched generalization test.

7. Quantitative Results and Evaluation Boundaries

7.1 Stage-wise results

Stage	Parameter count	A-OKVQA validation accuracy	Public weighted accuracy
Original model	4,022,969,088	79.00%	-
Structured pruning	2,978,586,976	76.60%	82.83%
Knowledge distillation	Same pruned architecture	-	85.73%
Final model with LoRA	2,986,746,208	-	86.59%

The A-OKVQA column reports the archived 1,000-example validation comparison, with a 2.40 percentage-point decrease after pruning. A dash means no verified value is supplied for that metric and stage. Public weighted accuracy uses a different dataset and metric, so comparisons must stay within a column. In particular, 86.59% cannot be compared with the original model's 79.00% as a gain on the same benchmark.

The final model is 25.76% smaller than the original, rounded to 25.8% in overview text. Before adapters, the pruned model is 25.96% smaller. The public gain over the pruned stage is 3.76 percentage points. These cumulative checkpoints do not isolate the causal contributions of KD, LoRA, and balanced sampling.

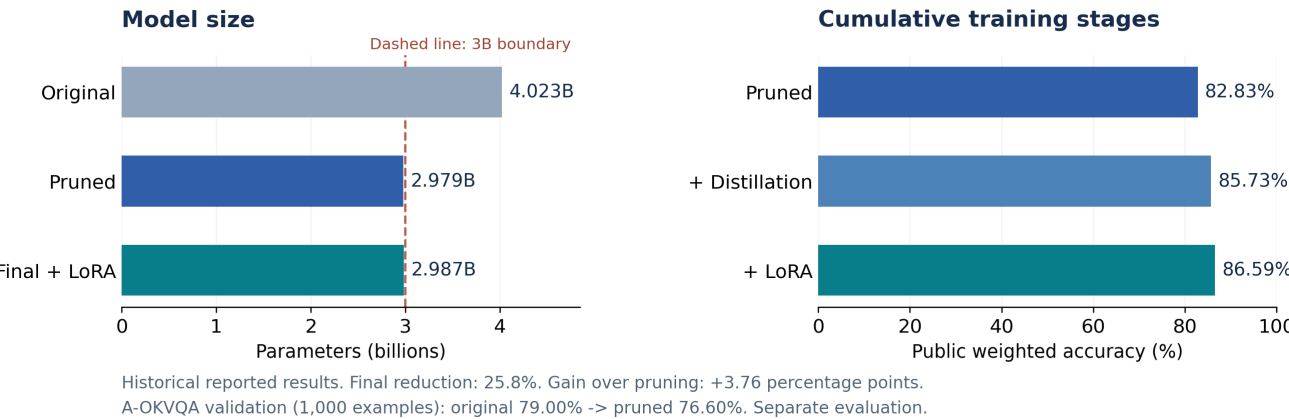


Figure 2. Archived parameter counts and public scores, with local A-OKVQA results stated separately. These are historical records; no new training or inference was run for the report.

7.2 Different subsets answered different questions

Use	Data	Role and limitation
Original/pruned comparison	A-OKVQA validation, 1,000 examples	Local baseline comparison
FFN calibration	First 250 A-OKVQA training examples	Importance signal; no random-sampling guarantee
Larger masking diagnostic	2,000 A-OKVQA training examples	Development sensitivity analysis
Head similarity	500 A-OKVQA training images, generic description prompt	Candidate signal with a prompt mismatch to multiple-choice VQA
Head-removal search	100 A-OKVQA validation examples	Small development subset; roughly 60 configurations
KD internal validation	10% of the 17,056-example A-OKVQA training split	Checkpoint selection by cross-entropy
LoRA training	7,010 A-OKVQA training examples	Balanced by preceding-model correctness

The subsets can overlap and should not be added as a count of unique examples. KD's internal holdout is distinct from the official validation split and is not guaranteed to remain held out from subsequent LoRA training. A separate 60-example competition diagnostic in the archive is not an A-OKVQA measurement and is not inserted into the stage table.

The public score and final award are distinct. Competition rules describe private-score evaluation and final presentation assessment [1]. Samsung's announcement documents the first-place award [2]; the public score alone did not define that outcome.

7.3 Historical resources

The archived summary lists a 32GB RTX 5090 for analysis and inference, an 80GB A100 for distillation at approximately seven hours, and a 24GB RTX 3090 for LoRA at approximately thirty minutes. These describe particular stages, not total experimentation cost or controlled latency, memory, or throughput measurements. The two-to-three-week development estimate and personal rental constraints are retrospective context.

8. What Task Accuracy Did Not Capture

8.1 Answer selection and language generation diverged

During development, I observed that a pruned model could still select correct answers while its ability to generate useful natural-language explanations deteriorated. I explored rationale training partly because this loss was visible despite acceptable answer behavior. This changed what I considered a complete evaluation of compression.

The observation is qualitative. A standardized explanation benchmark and systematic before-and-after error taxonomy are unavailable, and the exact form of the degradation is not fully reconstructed here. It does not establish that knowledge remained intact while only its expression was damaged. Fluent language would also not, by itself, establish a faithful account of the computation behind an answer.

Nevertheless, the experience exposed a concrete evaluation gap: preserving the metric used for selection did not ensure preservation of other useful behavior. It became a motivation for my interest in trustworthy AI and in measurements that reveal capability changes hidden by strong task scores.

8.2 How this experience informs my research questions

For systems that interact with people or act in the physical world, I want to understand whether efficiency and task-performance improvements preserve the capabilities people need to assess failures and intervene. The competition project did not measure robot safety, explanation faithfulness, or robustness in hazardous situations. It provided the experience from which these questions arose.

The question I would now ask is which capabilities changed, under which inputs, and which measurements would reveal an important loss before deployment. This interest follows from examining a limitation of my own successful system.

8.3 Limits of the experimental conclusions

The process involved one task setting, adaptive search, small subsets, and public feedback. There are no repeated-seed uncertainty estimates, complete ratio sweeps, matched-compute teacher comparisons, or factorial ablations in this release. Some recalled comparisons may involve several changes between runs. The archive also does not establish whether the pretrained model had encountered related data previously.

The evidence supports a working solution under concrete constraints and a record of how I used affordable observations to choose the next intervention. It does not establish universal superiority of one architecture, pruning order, teacher size, or sampling ratio.

9. Related Work and Retrospective Perspective

9.1 Methods used and intellectual credit

The system builds on InstructBLIP [3], A-OKVQA [4], Wanda's activation-and-weight principle [5], knowledge distillation [6], and LoRA [7]. Explaining how I arrived at an activation-and-output-weight question does not replace credit to Wanda. The recovery motivation explains my choice of distillation, rather than claiming that I originated teacher-student learning.

The following connections were made while contextualizing the work retrospectively. These papers should not be read as having guided the original experiments, nor as establishing that my project preceded similar ideas or that all my interpretations were correct.

9.2 Related research available before the competition

Gromov et al., The Unreasonable Ineffectiveness of the Deeper Layers, ICLR 2025 [8], show that some QA performance can survive layer removal while other evaluations deteriorate. This relates to capability-dependent sensitivity. Their language-model layer-removal setting differs from my InstructBLIP FFN/head compression and does not directly establish my explanation-generation observation.

Li et al., BLIP-2, ICML 2023 [9], demonstrate a Q-Former connecting frozen pretrained visual and language components. Maniparambil et al., Harnessing Frozen Unimodal Encoders for Flexible Multimodal Alignment, CVPR 2025 [10], study efficient alignment through projection modules and frozen encoders. These qualify my concern about pretraining-scale alignment costs: such costs are not necessary in every setting. The latter work primarily evaluates classification and retrieval, and its alignment savings reuse already-trained backbones. It does not guarantee inexpensive assembly of arbitrary generative VQA components.

Shu et al., LLaVA-MoD, ICLR 2025 [11], combine sparse MoE and distillation for smaller multimodal models. MoE remained a direction I wanted to investigate with more resources and eligible starting points. Fewer activated parameters alone would not meet this competition's total-loaded-parameter limit. Publication before the competition also does not

establish that released weights met its December 2023 cutoff.

9.3 Research published after the competition

Emmons et al., A Pragmatic Way to Measure Chain-of-Thought Monitorability, Google DeepMind, October 2025 preprint [12], distinguish whether reasoning is readable and whether it contains the steps needed to reach an answer. This supplies vocabulary for properties beyond correctness. Its proxy metrics concern reasoning traces and do not establish whether my model's monitorability changed.

Wen et al., SlimVLM, Huawei, August 2026 preprint [13], study VLM-specific structural pruning with module-dependent sensitivity and LoRA recovery. This relates to where capacity can be removed and how much performance can be recovered. Different models and criteria prevent treating it as validation of my exact removal rate, component ordering, or upstream-error hypothesis. Both corporate studies are identified as preprints rather than assumed conference publications.

10. Evidence Summary and Release Scope

Decision or observation	Evidence available	Scope of the claim
Independent solution development	Author account and official rules	Model/data selection through final implementation
Model selection for tractable experiments	Retrospective rationale; selected model in archive	No exhaustive architecture ranking
Encoder sensitivity and bridge concern	Recalled encoder trials; architectural hypothesis	No quantified Q-Former ablation
Conditional hidden-state masking	Archived diagnostics and recalled search	Exploratory; excluded from final architecture
FFN criterion and 95% removal	Inspected scoring and compaction code	Global neuron ranking; variable retained widths
Head similarity and removal set	Inspected attention/pruning notebooks	Original-model cosine signal; no measured head-FFN pairing
Original versus larger teacher	Original verified; alternative recalled	No numerical teacher-size ablation
KD settings	Final notebook and separate development logs	Configuration evidence; no fresh execution
Custom LoRA and 1:1 sampling	Inspected wrapper and selection code	Existing method adapted to compressed dimensions
Ratio and longer-training comparisons	Qualitative recollection	No fabricated sweep or split-specific curve
Language-generation degradation	Qualitative observation	No quantified faithfulness or safety conclusion
Stage scores and parameter counts	Archived summaries	Historical cumulative results
First-place award	Official Samsung announcement	Distinct from public leaderboard score

This public release contains the report, newly drawn figures, and aggregate tables. It excludes the competition presentation, private implementation notebooks, model checkpoints, and raw datasets. It is a technical project report rather than an executable reproduction package or a claim of peer-reviewed publication. Within that scope, it documents the reasoning, implementation decisions, outcomes, and remaining uncertainty behind the solution.

References

1. DACON. [SCPC 2025 AI Challenge: official rules](#). Competition constraints and evaluation procedure.
2. Samsung Research. [Samsung Electronics Unveils Winners of the 11th Samsung Collegiate Programming Challenge](#). Official announcement, 2025.
3. Dai et al. [InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning](#), NeurIPS 2023. [Selected model](#).
4. Schwenk et al. [A-OKVQA: A Benchmark for Visual Question Answering using World Knowledge](#), ECCV 2022. [Dataset repository](#).
5. Sun et al. [A Simple and Effective Pruning Approach for Large Language Models](#), ICLR 2024. [Wanda](#).
6. Hinton, Vinyals, and Dean. [Distilling the Knowledge in a Neural Network](#), 2015. Foundational method reference.
7. Hu et al. [LoRA: Low-Rank Adaptation of Large Language Models](#), ICLR 2022.
8. Gromov et al. [The Unreasonable Ineffectiveness of the Deeper Layers](#), ICLR 2025; preprint first released in 2024.
9. Li et al. [BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models](#), ICML 2023.
10. Maniparambil et al. [Harnessing Frozen Unimodal Encoders for Flexible Multimodal Alignment](#), CVPR 2025; preprint first released in 2024. [Author repository](#).
11. Shu et al. [LLaVA-MoD: Making LLaVA Tiny via MoE-Knowledge Distillation](#), ICLR 2025; preprint first released in 2024.
12. Emmons, Zimmermann, Elson, and Shah. [A Pragmatic Way to Measure Chain-of-Thought Monitorability](#), Google DeepMind, October 2025 preprint.
13. Wen et al. [SlimVLM: Sensitivity-aware Dynamic Structured Pruning with Adaptive Visual Token Selection for Efficient Vision-Language Models](#), Huawei, August 2026 preprint.